

ODOO DEVELOPER PROGRAM

Advanced Security

Odoo 17 Development Tutorials

Ahmed Muhumed

July 26, 2026

OUTLINE

- 1 Introduction
- 2 Security Groups
- 3 UI Visibility
- 4 More on Access Rights
- 5 Record Rules
- 6 Multi Company
- 7 Configuration Settings and System Parameters
- 8 Summary

Introduction

WHAT WILL WE LEARN?

- ▶ **Security Groups** – roles that users belong to
- ▶ **UI Visibility** – hiding menus, buttons, and fields by group
- ▶ **Access Rights** – model-level CRUD permissions
- ▶ **Record Rules** – row-level domain security
- ▶ **Multi Company** – company-aware security and data isolation
- ▶ **Configuration Settings & System Parameters** – admin settings and `ir.config.parameter`

WHY ADVANCED SECURITY MATTERS

- ▶ Beautiful views are useless if the wrong users can see or edit data.
- ▶ Odoo security is layered:
 - 1 Groups
 - 2 Access rights (model)
 - 3 Record rules (records)
 - 4 UI visibility (menus/views/buttons)
 - 5 Multi-company isolation
- ▶ Good security design protects data and improves UX.

SECURITY LAYERS AT A GLANCE

- ▶ **Group:** Who is the user? (Teacher, Manager, Officer)
- ▶ **Access right:** What can that group do on a model?
(read/write/create/unlink)
- ▶ **Record rule:** Which records are visible/editable?
- ▶ **UI rules:** Which menus/buttons/fields appear?
- ▶ **Company:** Which company does the record belong to?

Security Groups

WHAT IS A SECURITY GROUP?

- ▶ A **security group** is a role stored in `res.groups`.
- ▶ Users are assigned to one or more groups.
- ▶ Groups control access rights, record rules, and UI visibility.

Core Idea

Give permissions to groups, then assign users to groups – not the opposite.

DEFINING A CATEGORY AND GROUPS

```
1 <record id="module_category_school" model="ir.module.category">
2   <field name="name">School</field>
3   <field name="sequence">20</field>
4 </record>
5
6 <record id="group_school_user" model="res.groups">
7   <field name="name">User</field>
8   <field name="category_id" ref="module_category_school"/>
9   <field name="implied_ids" eval="[(4, ref('base.group_user'))]"/>
10 </record>
11
12 <record id="group_school_manager" model="res.groups">
13   <field name="name">Manager</field>
14   <field name="category_id" ref="module_category_school"/>
15   <field name="implied_ids" eval="[(4, ref('group_school_user'))]"/>
16   <field name="users" eval="[(4, ref('base.user_root')), (4, ref('base.user_admin'))]"/>
17 </record>
```

BREAKING DOWN GROUP FIELDS

- ▶ **name:** group label (User, Manager, Officer)
- ▶ **category_id:** Settings app section for the group
- ▶ **implied_ids:** groups automatically included
- ▶ **users:** optional default members
- ▶ **comment:** description of the role

```
1 <field name="implied_ids" eval="[(4, ref('group_school_user'))]" />
```

- ▶ Manager implies User $\Rightarrow$ Manager gets all User rights too.

TYPICAL SCHOOL PRIVILEGE LADDER

- 1 **User:** read students, limited write
- 2 **Officer/Teacher:** create/update operational data
- 3 **Manager:** full CRUD + configuration

```
1 <!-- Manager implies Officer implies User -->  
2 <field name="implied_ids" eval="[(4, ref('group_school_officer'))]" />
```

► Design roles as a ladder, not disconnected islands.

CHECKING GROUPS IN PYTHON

```
1 if self.env.user.has_group( 'school_manager.group_school_manager' ):
2     # manager-only logic
3     pass
4
5 if not self.env.user.has_group( 'school_manager.group_school_user' ):
6     raise AccessError( "You need School User access." )
```

- ▶ Prefer XML-ID group checks over hard-coded group IDs.
- ▶ Still define real ACL/record rules; Python checks are extra safety.

GROUP DESIGN BEST PRACTICES

- ▶ Create a module category for your app.
- ▶ Keep group names short and clear.
- ▶ Use `implied_ids` for privilege inheritance.
- ▶ Assign rights to groups, not individual users.
- ▶ Put group XML in `security/` and load it before access CSV.

UI Visibility

WHAT IS UI VISIBILITY?

- ▶ UI visibility controls what the user **sees** in the interface.
- ▶ Even if a user has model access, you may hide:
 - menus
 - buttons
 - fields
 - pages / groups inside forms
- ▶ Visibility is usually based on groups.

RESTRICTING MENUS BY GROUPS

```
1 <menuItem id="menu_student_root"  
2   name="Students"  
3   groups="school_manager.group_school_user"/>  
4  
5 <menuItem id="menu_school_config"  
6   name="Configuration"  
7   parent="menu_student_root"  
8   groups="school_manager.group_school_manager"/>
```

- ▶ Users outside the group do not see the menu.
- ▶ Configuration menus usually belong to managers only.

HIDING BUTTONS AND FIELDS

```
1 <button name="action_approve"  
2   type="object"  
3   string="Approve"  
4   groups="school_manager.group_school_manager"/>  
5  
6 <field name="internal_note"  
7   groups="school_manager.group_school_manager"/>
```

- ▶ groups= on view nodes is the most common UI restriction.
- ▶ Hidden UI is not enough by itself – still protect with ACL/record rules.

INVISIBLE / READONLY MODIFIERS

```
1 <field name="approved_by"  
2   readonly="1"  
3   groups="school_manager.group_school_manager"/>  
4  
5 <field name="salary_level"  
6   invisible="state != 'done'"  
7   groups="school_manager.group_school_manager"/>
```

- ▶ groups: who can see the node at all
- ▶ invisible / readonly: dynamic UI behavior
- ▶ In Odoo 17, prefer Python-like modifier expressions.

RESTRICTING ACTIONS

```
1 <record id="action_student" model="ir.actions.act_window">
2   <field name="name">Students</field>
3   <field name="res_model">student.student</field>
4   <field name="view_mode">list,form</field>
5   <field name="groups_id" eval="[(4, ref('group_school_user'))]" />
6 </record>
```

- ▶ Actions and menus can both be group-restricted.
- ▶ This prevents opening screens through direct action links.

UI VISIBILITY BEST PRACTICES

- ▶ Hide advanced controls from normal users.
- ▶ Keep forms clean: show only what each role needs.
- ▶ Never rely on hidden buttons as your only security.
- ▶ Combine UI groups + access rights + record rules.
- ▶ Test each role with a real non-admin user.

More on Access Rights

WHAT ARE ACCESS RIGHTS?

- ▶ Access rights define model-level permissions in `ir.model.access`.
- ▶ For each group + model, you allow or deny:
 - `perm_read`
 - `perm_write`
 - `perm_create`
 - `perm_unlink`

Usual File

1 `security/ir.model.access.csv`

ACCESS RIGHTS CSV IN DETAIL

```
1 id ,name,model_id:id ,group_id:id ,perm_read ,perm_write ,perm_create ,perm_unlink
2 access_student_user ,student.user,model_student_student ,school_manager .
   group_school_user ,1,0,0,0
3 access_student_officer ,student.officer ,model_student_student ,school_manager .
   group_school_officer ,1,1,1,0
4 access_student_manager ,student.manager,model_student_student ,school_manager .
   group_school_manager ,1,1,1,1
```

BREAKING DOWN CSV COLUMNS

- ▶ **id**: external ID of the access line
- ▶ **name**: label
- ▶ **model_id:id**: model XML ID (`model_student_student`)
- ▶ **group_id:id**: group XML ID
- ▶ **perm_***: 1 allow, 0 deny

1 `model_student_student`

- ▶ Odoo creates that XML ID from `model student.student`.

GLOBAL ACCESS VS GROUP ACCESS

```
1 id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm.unlink  
2 access_student_all,student.all,model_student_student,,1,0,0,0
```

- ▶ Empty `group_id` means the right applies to everyone.
- ▶ Use sparingly; prefer explicit group rights.

ACCESS RIGHTS FOR TRANSIENT / WIZARD MODELS

```
1 access_assign_wizard_user , assign.wizard.user , model_student_assign_wizard ,  
   school_manager.group_school_user , 1 , 1 , 1 , 1
```

- ▶ Wizards also need ACL lines.
- ▶ Users often need create/write/read on transient models to open and submit wizards.
- ▶ Forgetting wizard ACL is a common “Access Error” cause.

TESTING ACCESS IN PRACTICE

```
1 # Switch to a non-admin user in shell / tests
2 Student = self.env[ 'student.student' ].with_user(teacher)
3 Student.check_access_rights( 'read ' )
4 Student.check_access_rights( 'unlink ' ) # may raise
```

- ▶ Always verify rights with a normal user, not only admin.
- ▶ Admin/superuser can hide ACL mistakes.

ACCESS RIGHTS BEST PRACTICES

- ▶ Give least privilege needed for each role.
- ▶ Managers get unlink; normal users usually do not.
- ▶ Load security XML groups before the ACL CSV.
- ▶ Add ACL for every custom model, including wizards.
- ▶ Document your role matrix (User/Officer/Manager × CRUD).

Record Rules

WHAT ARE RECORD RULES?

- ▶ Record rules provide **row-level security**.
- ▶ Stored in `ir.rule`.
- ▶ They filter which records a group can read/write/create/unlink.

Core Idea

Access rights say “you may read students”.

Record rules say “but only students in your classroom”.

BASIC RECORD RULE XML

```
1 <record id="student_personal_rule" model="ir.rule">
2   <field name="name">Students: own classroom only</field>
3   <field name="model_id" ref="model_student_student"/>
4   <field name="domain_force">[(('classroom_id.teacher_id.user_id', '=', user.id)]</field>
5   <field name="groups" eval="[(4, ref('group_school_user'))]">/>
6   <field name="perm_read" eval="True"/>
7   <field name="perm_write" eval="True"/>
8   <field name="perm_create" eval="True"/>
9   <field name="perm_unlink" eval="False"/>
10 </record>
```

BREAKING DOWN RECORD RULE FIELDS

- ▶ **name**: clear description of the rule
- ▶ **model_id**: target model
- ▶ **domain_force**: domain filter
- ▶ **groups**: which groups the rule applies to
- ▶ **perm_***: on which operations the rule applies
- ▶ If `groups` is empty, the rule is global

USEFUL DOMAIN VARIABLES

```
1 <field name="domain_force">[(user_id, '=', user.id)]</field>  
2 <field name="domain_force">[(company_id, 'in', company_ids)]</field>  
3 <field name="domain_force">[|, (teacher_id, '=', False), (teacher_id.user_id, '=', user.id)]</field>
```

► Common variables in rule domains:

- user
- company_id
- company_ids
- time / helpers depending on version/context

GLOBAL RULES VS GROUP RULES

- ▶ **Group rule:** applies only to listed groups.
- ▶ **Global rule:** no groups set; applies to everyone.

```
1 <!-- Global example: everyone restricted by company -->
2 <record id="student_company_rule" model="ir.rule">
3   <field name="name">Students: multi-company</field>
4   <field name="model_id" ref="model_student_student"/>
5   <field name="domain_force">[('company_id', 'in', company_ids)]</field>
6 </record>
```

HOW RULES COMBINE

- ▶ For the same model:
 - Global rules are combined with AND
 - Group rules for the user are combined with OR, then AND-ed with globals
- ▶ Practical meaning:
 - Global rules always restrict
 - Group rules can open alternative allowed sets for that role

RECORD RULES VS ACCESS RIGHTS

- ▶ No read access right → model blocked completely.
- ▶ Read access + record rule → only matching rows are visible.

```
1 # User can read model, but sees only allowed rows  
2 students = self.env['student.student'].search([])
```

- ▶ Use both layers together.

RECORD RULE BEST PRACTICES

- ▶ Name rules by business meaning, not technical noise.
- ▶ Keep domains simple and indexed when possible.
- ▶ Add a manager bypass rule when managers must see all records.
- ▶ Test with multiple users in different groups.
- ▶ Remember: `sudo ()` bypasses record rules – use carefully.

Multi Company

WHAT IS MULTI-COMPANY?

- ▶ Multi-company allows one database to serve multiple companies.
- ▶ Records often belong to a `company_id`.
- ▶ Users may have access to one or several companies.

Typical Field

```
1 company_id = fields.Many2one(  
2     'res.company',  
3     required=True,  
4     default=lambda self: self.env.company,  
5 )
```

COMPANY FIELD ON BUSINESS MODELS

```
1 class Student(models.Model):  
2     _name = 'student.student'  
3  
4     name = fields.Char(required=True)  
5     company_id = fields.Many2one(  
6         'res.company',  
7         string='Company',  
8         required=True,  
9         index=True,  
10        default=lambda self: self.env.company,  
11    )
```

- ▶ Default to `env.company`.
- ▶ Index `company_id` because security domains use it often.

MULTI-COMPANY RECORD RULE

```
1 <record id="student_comp_rule" model="ir.rule">
2   <field name="name">Student multi-company</field>
3   <field name="model_id" ref="model_student_student"/>
4   <field name="domain_force">[(company_id, 'in', company_ids)]</field>
5 </record>
```

- ▶ `company_ids` = companies currently allowed in the user environment.
- ▶ This is the standard isolation pattern.

ENVIRONMENT HELPERS FOR COMPANY

```
1 # Active company
2 company = self.env.company
3
4 # Allowed companies
5 companies = self.env.companies
6
7 # Force operations in another company
8 records = self.env[ 'student.student' ].with_company( other_company ).search ( [])
```

- Prefer `with_company()` over hard-coding company IDs.

VIEWS AND COMPANY FIELDS

```
1 <field name="company_id" groups="base.group_multi_company"/>
```

- ▶ Often show `company_id` only when multi-company is enabled.
- ▶ In many2one fields, domain can restrict to current companies:

```
1 <field name="classroom_id"  
2   domain="[(company_id, 'in', company_ids)]"/>
```

CHECK COMPANY CONSISTENCY

```
1 @api.constrains('classroom_id', 'company_id')
2 def _check_company_consistency(self):
3     for student in self:
4         if student.classroom_id and student.classroom_id.company_id != student.
           company_id:
5             raise ValidationError(
6                 "Student and Classroom must belong to the same company."
7             )
```

- Related records should usually share the same company.

MULTI-COMPANY BEST PRACTICES

- ▶ Add `company_id` early on shared business models.
- ▶ Add a company record rule.
- ▶ Default from `env.company`.
- ▶ Validate consistency between related records.
- ▶ Test with users who can access more than one company.

Configuration Settings and System Parameters

WHAT ARE CONFIGURATION SETTINGS?

- ▶ Configuration settings are admin options shown in Settings.
- ▶ In modern Odoo they usually extend `res.config.settings`.
- ▶ They let managers enable features and store defaults without code changes.

Model

```
1 class ResConfigSettings(models.TransientModel):  
2     _inherit = 'res.config.settings'
```

ADDING SETTINGS FIELDS

```
1 class ResConfigSettings(models.TransientModel):  
2     _inherit = 'res.config.settings'  
3  
4     school_default_age = fields.Integer(  
5         string='Default Student Age',  
6         config_parameter='school_manager.default_age',  
7     )  
8     school_allow_self_register = fields.Boolean(  
9         string='Allow Self Registration',  
10        config_parameter='school_manager.allow_self_register',  
11    )
```

► `config_parameter` stores the value in `ir.config_parameter`.

SETTINGS VIEW XML

```
1 <record id="res.config.settings.view.form.school" model="ir.ui.view">
2   <field name="name">res.config.settings.view.form.school</field>
3   <field name="model">res.config.settings</field>
4   <field name="inherit_id" ref="base.res.config.settings.view.form"/>
5   <field name="arch" type="xml">
6     <xpath expr="//form" position="inside">
7       <app string="School" name="school_manager">
8         <block title="Student Defaults">
9           <setting string="Default Age">
10             <field name="school_default_age"/>
11           </setting>
12           <setting string="Self Registration">
13             <field name="school_allow_self_register"/>
14           </setting>
15         </block>
16       </app>
17     </xpath>
18   </field>
19 </record>
```

WHAT IS `IR.CONFIG_PARAMETER`?

- ▶ System parameters are key/value settings stored in `ir.config_parameter`.
- ▶ Useful for module-wide options and integrations.

```
1 ICP = self.env[ 'ir.config_parameter' ].sudo()  
2 age = int(ICP.get_param( 'school_manager.default_age', default=18))  
3 ICP.set_param( 'school_manager.allow_self_register', True)
```

- ▶ Often accessed with `sudo()` because parameters are technical.

CREATING A PARAMETER IN XML

```
1 <record id="school_default_age_param" model="ir.config.parameter">
2   <field name="key">school_manager.default_age</field>
3   <field name="value">18</field>
4 </record>
```

- ▶ Good for default install values.
- ▶ Prefer namespaced keys: `module_name.option_name`.

USING SETTINGS IN BUSINESS CODE

```
1 @api.model
2 def _default_age(self):
3     value = self.env['ir.config_parameter'].sudo().get_param(
4         'school_manager.default_age', '18'
5     )
6     return int(value)
7
8 age = fields.Integer(default=_default_age)
```

- ▶ Settings become real defaults for models and flows.
- ▶ Keep conversion/validation around string parameters.

SECURITY AROUND SETTINGS

```
1 <menuitem id="menu_school_settings"  
2   name="Settings"  
3   parent="menu_school_root"  
4   action="action_school_config_settings"  
5   groups="base.group_system"/>
```

- ▶ Settings pages are usually for Administration / Settings users.
- ▶ Do not expose technical parameters to normal employees.

SETTINGS BEST PRACTICES

- ▶ Use `res.config.settings` for admin-facing options.
- ▶ Store durable values with `config-parameter`.
- ▶ Namespace keys by module.
- ▶ Provide sensible defaults in XML or field defaults.
- ▶ Restrict settings UI to the right group.

Summary

ADVANCED SECURITY SUMMARY

- ▶ **Groups** define roles and privilege ladders.
- ▶ **UI visibility** hides menus/buttons/fields per group.
- ▶ **Access rights** control model-level CRUD.
- ▶ **Record rules** filter which rows each role can reach.
- ▶ **Multi-company** isolates data through `company_id` and company rules.
- ▶ **Settings / system parameters** store admin configuration safely.

SECURITY DESIGN CHECKLIST

- 1 Create groups and category
- 2 Add ACL CSV for every model/wizard
- 3 Add record rules for row ownership/company
- 4 Restrict menus/buttons/fields with `groups=`
- 5 Add company fields/rules if multi-company
- 6 Put admin options in Settings + `ir.config_parameter`
- 7 Test with non-admin users for each role

SUGGESTED SECURITY FILES

```
1 school_manager/  
2   security/  
3     school_security.xml      # categories, groups, record rules  
4     ir.model.access.csv      # ACL  
5   views/  
6     res_config_settings_views.xml  
7   data/  
8     ir_config_parameter_data.xml
```

FINAL REMINDER

- ▶ Hidden UI $\neq$ secure data.
- ▶ Always combine:
 - Groups
 - Access rights
 - Record rules
- ▶ Admin/sudo can hide bugs – verify with real role users.